MMD V2 - System Architecture

1. Architecture Overview

Architecture Type:

Modular Monolith

Why?

- Solo Developer Friendly
- Easier Deployment
- Faster Development
- Lower Cost
- Easier Maintenance

The system must remain modular so it can evolve into microservices if required.

---

2. Technology Stack

Frontend

- Next.js 15
- TypeScript
- TailwindCSS
- ShadCN UI

Backend

- Next.js Route Handlers
- Server Actions

Database

- PostgreSQL

ORM

- Prisma

Authentication

- Auth.js

Validation

- Zod

Documentation

- OpenAPI
- Swagger
- Postman

Deployment

- Docker

CI/CD

- GitHub Actions

Logging

- Pino

Monitoring

- BetterStack

---

3. High Level Architecture

Browser

↓

Next.js App

↓

API Layer

↓

Service Layer

↓

Repository Layer

↓

Prisma ORM

↓

PostgreSQL

---

4. Project Layers

Presentation Layer

Responsibilities:

- Pages
- Components
- Forms
- UI Logic

Application Layer

Responsibilities:

- Business Logic
- Use Cases
- Workflows

Infrastructure Layer

Responsibilities:

- Database
- Storage
- Email
- Logging

Domain Layer

Responsibilities:

- Business Rules
- Entities
- Validation

---

5. Request Flow

User Request

↓

Middleware

↓

Authentication

↓

Tenant Validation

↓

Permission Validation

↓

API Route

↓

Service

↓

Repository

↓

Prisma

↓

PostgreSQL

↓

Response

---

6. Feature Based Architecture

features/

auth/

users/

roles/

companies/

contacts/

leads/

requirements/

candidates/

applications/

interviews/

placements/

timesheets/

invoices/

templates/

workflows/

reports/

settings/

Each feature owns:

- API
- Validation
- Service
- Repository
- Types

---

7. Repository Pattern

Never call Prisma directly from routes.

Wrong:

Route → Prisma

Correct:

Route → Service → Repository → Prisma

Benefits:

- Testability
- Database Independence
- Reusability

---

8. Service Layer

Purpose:

Business Logic

Example:

Convert Lead

Responsibilities:

- Validate lead
- Create requirement
- Update status
- Create audit log

- Send notification

This logic must not live inside API routes.

---

9. Authentication Architecture

Auth.js

↓

Session

↓

Middleware

↓

Protected Routes

User Login

↓

Validate Credentials

↓

Generate Session

↓

Store Session

↓

Redirect

---

10. Authorization Architecture

RBAC

↓

Permission Matrix

↓

Middleware

↓

API Access

Example:

companies.create

↓

Verify Permission

↓

Allow Request

---

11. Multi Tenant Architecture

Shared Database

Shared Schema

Tenant Isolation

Every table contains:

tenantId

Every query automatically filters:

tenantId

Example:

WHERE tenantId = currentTenant

No cross-tenant access.

---

12. File Storage Architecture

Development

Local Storage

Production

S3 Compatible Storage

Options:

- AWS S3
- Cloudflare R2
- MinIO

Recommended:

Cloudflare R2

Low Cost

---

13. Email Architecture

Provider Layer

Email Service Interface

↓

SMTP Provider

Options:

- Resend
- SendGrid
- SMTP

Recommended:

Resend

---

14. Notification Architecture

System Event

↓

Notification Service

↓

Email

↓

In-App Notification

Future:

WhatsApp

SMS

---

15. Audit Logging

Every Action

↓

Audit Service

↓

audit_logs table

Track:

Create

Update

Delete

Login

Permission Changes

---

16. Search Architecture

Global Search

↓

Search Service

↓

Multiple Repositories

Search:

Companies

Leads

Requirements

Candidates

Placements

---

17. Queue Architecture

Phase 1

Database Queue

jobs table

Worker Process

↓

Process Jobs

No Redis

No BullMQ

Phase 2

Redis

+

BullMQ

Only when scale demands it.

---

18. Reporting Architecture

Operational Tables

↓

Report Service

↓

Aggregated Queries

↓

Dashboard

Future:

Analytics Warehouse

---

19. Integration Architecture

Inbound

Webhooks

Outbound

Webhooks

API Keys

Supported Integrations:

CRM

HRMS

Job Boards

Email Platforms

---

20. Error Handling Architecture

Error

↓

Logger

↓

API Response

↓

User Notification

Never expose:

Stack Traces

Database Errors

Secrets

---

21. Logging Architecture

Pino Logger

↓

Log Service

↓

Database

↓

BetterStack

Track:

Errors

Warnings

Audit

Performance

---

22. Monitoring Architecture

Monitor:

API Latency

Database Queries

Error Rate

Job Queue

Storage

Health Checks

---

23. Security Architecture

Authentication

Authorization

Tenant Isolation

Input Validation

Rate Limiting

CSRF Protection

XSS Protection

Audit Logs

Encrypted Secrets

---

24. Deployment Architecture

Developer Machine

↓

Docker

↓

Staging

↓

Production

Single Container Deployment Initially

---

25. Future Scalability

Phase 1

Single Application

↓

Phase 2

Separate Worker

↓

Phase 3

Read Replicas

↓

Phase 4

Microservices (if needed)

Never start with microservices.

---

26. Architectural Principles

Business Logic in Services

Database Access in Repositories

Validation with Zod

Tenant Isolation Everywhere

API First Design

Reusable Components

Feature Based Structure

Minimal Infrastructure

Cost Efficient Scaling

Developer Friendly

Enterprise Ready